US Housing and Community Data Explorer

ZIP/ZCTA Housing, Rent, and Income Analysis

DSCI 551 Project Proposal

Fall 2026

Samira Kazimi and Shusei Yokoi

We propose to build a MySQL database that integrates ZIP level Zillow home values, rents, and household income across the United States. The project will support housing searches and comparisons. Its main academic focus will be relational database design, data integration, meaningful SQL queries, indexing, and query performance analysis.

A. Team and responsibilities

We will complete this project as a two member team. Both members will review the complete system and contribute to the final report and demonstration.

Samira Kazimi: Lead source documentation, csv and API downloads, basic data cleaning and reshaping, ZIP code standardization, and the data quality log. Prepare cleaned files for loading, build the basic interface, and draft the motivation, data, and user interface sections of the report.

Shusei Yokoi: Lead relational schema refinement, MySQL table creation and constraints, reproducible loading and transaction handling Design and automate the index experiments, compare multiple physical designs and query conditions, interpret execution plans, and measure performance.

Shared responsibilities: Agree on meaningful application questions and the cleaned-data format early. Review the ER diagram, schema, SQL results, and performance findings together. Validate the complete application, combine and edit the report, and rehearse the demonstration.

B. Problem and motivation

- Application/domain: Our project focuses on housing and real estate. We will develop a MySQL database that integrates ZIP level home values, rents, and household income across the United States. Crime data may be added if a suitable source is available.

- Problem to address: Housing and community data come from different sources and use different formats. These differences make it difficult to compare ZIP codes using consistent queries. We will organize the data in related tables, connect them through ZIP identifiers, and document differences in coverage and reporting periods.

- Why the data are interesting: These data allow us to explore how housing values and rents vary across ZIP codes and change over time. Combining housing and income data also provides useful context for comparing regional housing costs. The different sources create meaningful opportunities to study data integration, relational database design, and query performance.

- Intended users and use cases: Intended users include people researching housing markets and students studying regional housing conditions. Users will be able to search by location, compare home values and rents, examine historical trends, and explore housing costs in relation to ZIP income. ZIP level statistics will describe regional conditions; they will not represent individual properties or a person’s purchasing ability.

C. Data sources

We will use the Zillow Home Value Index (ZHVI), the Zillow Observed Rent Index (ZORI), and the U.S. Census Bureau's ACS API for ZIP-level home values, rents, and median household income across the 50 states and Washington, DC. Housing and rent data will cover January 2015 through December 2025, and income will use the 2024 ACS five-year release (2020–2024).

- URL or documentation:

- Zillow housing and rental data: https://www.zillow.com/research/data/
- Census 2024 ACS five year dataset: https://api.census.gov/data/2024/acs/acs5.html
- Census income variable documentation: https://api.census.gov/data/2024/acs/acs5/variables/B19013_001E.html
- Census API documentation and query limits: https://www.census.gov/data/developers/guidance/api-user-guide/query-components.html

- Type and format of the data:

Zillow provides structured csv files containing ZIP information and monthly observations. We will use one smoothed, seasonally adjusted ZIP series for home values and one for rents. We will transform the monthly columns into rows, with one observation for each ZIP code and month.

The Census API returns structured JSON data. We will convert the response into tabular records before loading it into MySQL. The ACS income estimates represent a five year period, so we will clearly label their reporting period when comparing them with monthly housing data.

- Important fields:

For the Zillow datasets, important fields include:

- RegionID: Zillow's identifier for a geographic region.
- RegionName: 5-digit ZIP code.
- State: state abbreviation.
- City and CountyName: descriptive attributes for each ZIP code.
- Monthly date columns: home value or rent observations.

For the Census dataset, important fields include:

- NAME: geographic area name.
- B19013_001E: median household income estimate.
- B19013_001M: margin of error for the income estimate.
- zip code tabulation area: 5-digit ZCTA code.

- Approximate expected data volume:

We expect approximately 3.3 million nonempty home-value observations from about 26,000 ZIP code rows and 700,000 rent observations from about 7,500 ZIP code rows for 2015–2025. We also expect approximately 33,800 income records from the ACS API, reported by ZIP Code Tabulation Area (ZCTA). The combined dataset is expected to contain about 4 million observations before final validation. Coverage differs across sources, and

many ZIP codes have missing months, especially for rents. We will report the final record counts and coverage after cleaning and loading the data.

- API restrictions or rate limits:

The Census API documentation allows up to 50 variables per request and 500 requests per IP address per day without an API key. We will cache downloaded responses and store any API key in an environment variable.

The Zillow csv downloads did not require an API key during the preliminary checks. A specific numeric download limit has not been confirmed. We will avoid repeated downloads and review the applicable data use terms before redistributing data or using them in a public application.

- Planned data collection method:

We will write new Python scripts to download the Zillow csv files and request income data through the Census API. We will preserve the original files and record their source URLs and download dates.

During preprocessing, we will reshape the monthly housing data, standardize ZIp codes, filter the required dates and geographic areas, and check for duplicate records and invalid values. Census missing value codes will be converted to NULL. Missing housing observations will be documented and will not be treated as zero. We will then load the validated records into MySQL using a reproducible loading process. The process will check primary key uniqueness and foreign key relationships and prevent duplicate records when the same dataset is loaded again. Geographic differences and unmatched ZIP codes will be documented.

D. Preliminary database design

- Major entities:

The database will contain six main entities:

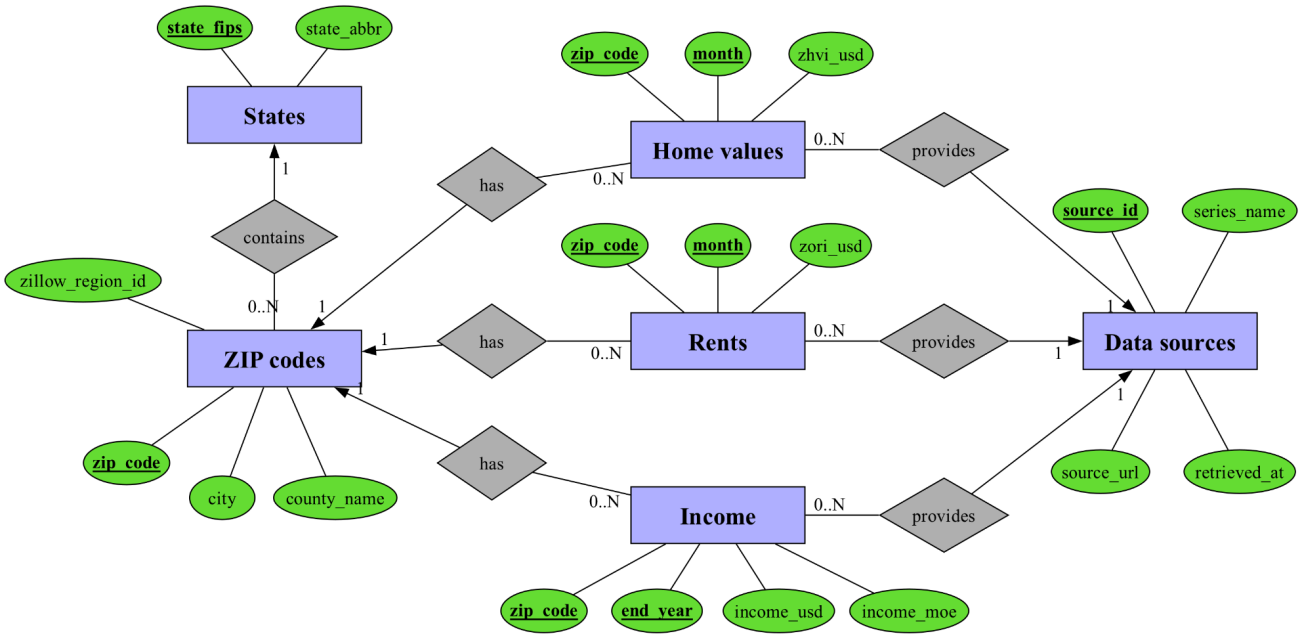
1. States: Stores state identifiers and abbreviations, including Washington, DC, as a state equivalent area.
2. Data sources: Stores information about the origin of the data, including the source name, URL, and retrieval date.
3. ZIP Codes: Stores 5-digit ZIP/ZCTA codes, city and county names, and the associated state
4. Home values: Stores monthly ZIP level home values from Zillow ZHVI.
5. Rents: Stores monthly ZIP level rents from Zillow ZORI.
6. Income: Stores ZIP level median household income estimates and their margins of error from the Census ACS.

Separating these entities will reduce repeated information and support consistent queries across the datasets.

- Important relationships:

Each ZIP belongs to one state or state equivalent area. Each state can contain many ZIP codes, creating a one to many relationship. Each ZIP can have many home value, rent, and income observations. Each observation belongs to one ZIP, creating additional one to many relationships. ZIP codes with missing data may have no observations in a particular table. Each observation also refers to one data source. A data source can provide many observations. Foreign keys will maintain these relationships and prevent observations from referring to ZIP codes or sources that do not exist in the database.

Preliminary ER diagram



Initial relational schema

The initial schema will use the following tables and keys:

Table	Primary key	Foreign keys
states	state_fips	None
zip_codes	zip_code	state_fips references states
data_sources	source_id	None
home_values	(zip_code, month)	zip_code references zip_codes, source_id references data_sources
rents	(zip_code, month)	zip_code references zip_codes, source_id references data_sources
income	(zip_code, end_year)	zip_code references zip_codes, source_id references data_sources

The relational schema will define data types, keys, and constraints. ZIP codes and state FIPS codes use CHAR to preserve leading zeros, monetary values use DECIMAL, and monthly dates use DATE. Composite primary keys prevent duplicate zip-month and zip-year observations. Missing values may be NULL, while CHECK constraints validate numeric values. States, ZIP codes, and sources will be loaded first, followed by observations using transactions and duplicate prevention. Zillow reports by USPS ZIP code and the ACS by ZCTA; we will match on the 5-digit code, load the union of both into zip_codes, and log codes that appear in only one source.

E. Proposed Query Workload:

The database will answer the following application questions:

1. Which ZIP codes in a selected state and month have typical home values below a user selected amount, ordered from lowest to highest?
2. How have monthly home values changed for a selected zip during a chosen date range?
3. Which ZIP codes have the largest percentage increase in home values between two selected months?
4. What are the typical home values and rents for selected ZIP codes during the same month?
5. How do December 2024 home values compare with median household income from the 2020–2024 ACS period across ZIP code?
6. For a selected month, what is the unweighted average ZIP home value in each state, and how many ZIP codes are included?

These questions will involve selection, filtering, multi table joins, aggregation, GROUP BY, ORDER BY, and queries that may benefit from indexes.

The home value to income ratio will be an illustrative regional indicator, not a measure of an individual's mortgage eligibility.

F. Potential Physical-Design Opportunities:

- Indexing: For a query filtering by month and home-value range and ordering by value, we will compare the baseline against indexes on (month), (zhvi_usd), (month, zhvi_usd, zip_code), and (zhvi_usd, month, zip_code) in home_values, explaining how column order and selectivity affect the optimizer's choices.

- Sorting: We will check whether MySQL sorts explicitly or reads orders from an index when ordering by home value in one month, by date for one zip code, and by calculated percentage growth.

- Joins: We will join home_values and rents on zip_code and month, comparing zip-first primary keys with added (month, zip_code) indexes over a single month and a longer date range.

- Query performance: We will use EXPLAIN and/or EXPLAIN ANALYZE to compare plans and row counts across narrow and broad value ranges. Using the same data and parameters, we will report the median of at least three warmed-up runs, document the test environment, and explain improvements, slowdowns, and unused indexes.

G. Project Scope:

We will complete this project as a two-member team using three public datasets (one home value series, one rent series, and one ACS five-year income release), six relational tables, and approximately 480,000 expected observations. The scope includes reproducible data acquisition, MySQL implementation, a meaningful query workload, and a simple database-connected interface, with crime integration remaining optional and attempted only after the core system and performance experiments are complete. To meet two-member expectations, we will pursue the deeper MySQL analysis option instead of adding MongoDB or Firebase, comparing single-column and composite indexes, alternative column orders, and narrow versus broad filtering and date-range conditions, then connecting the measured results and query plans to indexing, sorting, joins, and predicate selectivity. The final deliverables will include acquisition and loading scripts, the relational schema, SQL queries, index experiments, execution plans, performance measurements, a working interface, a demonstration, and the final report, with code and reproducibility materials organized by Samira in a Google Drive folder linked in the report. Each member will document their contribution, all core functionality will be ready before our assigned demonstration.